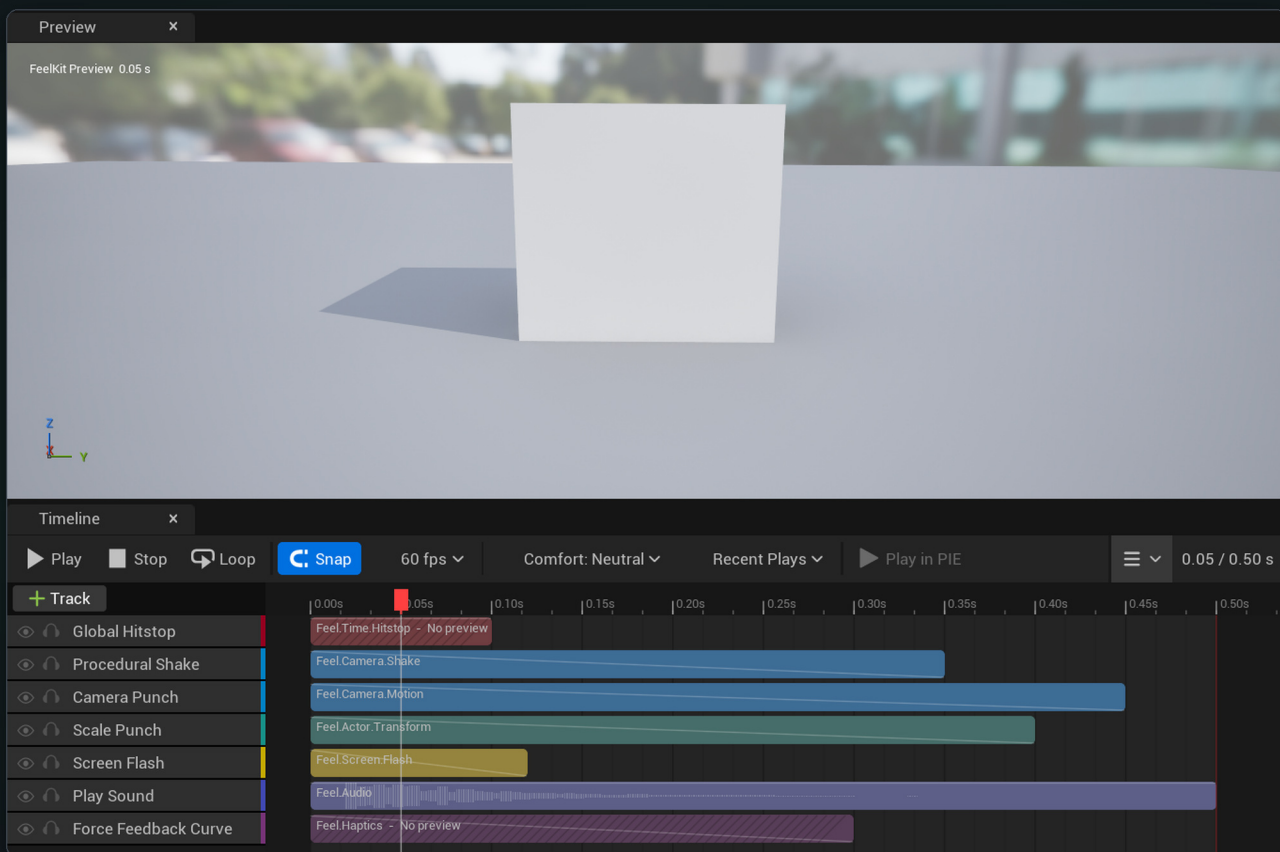


UNREAL ENGINE PLUGIN

FeelKit

Manual

Build game feel on a timeline. Play it with one node.
Let every player choose how much they feel.



TIMELINE

Build and preview a hit without pressing Play.

ONE NODE

Play recipes from Blueprint or C++, or send an event.

COMFORT

Every player chooses how much shake, flash and rumble they get.

Contents

- 2 Core ideas..... 4
 - 2.1 Recipes, tracks and steps 4
 - 2.2 Channels, and what happens when effects overlap 5
- 4 Quick start: your first recipe 7
 - 4.1 Pick a recipe from the library 7
 - 4.2 Preview it without pressing Play 8
 - 4.3 Play it from a Blueprint 9
 - 4.4 Compare with the feel off..... 11
 - 4.5 Where to go next 12
- 16 Step reference 14
 - 16.5 Camera steps..... 14

PRO In FeelKit Pro only. Everything without this label is in FeelKit Lite and FeelKit Pro.

Draft, 25 September 2026

PART ONE

Using FeelKit

- 1 Welcome
- 2 Core ideas
- 3 Installing FeelKit
- 4 Quick start: your first recipe
- 5 The recipe editor
- 6 Playing recipes in your game
- 7 Parameters, context and variation
- 8 Comfort and accessibility
- 9 The recipe library and browser
- 10 Debugging and tuning
- 11 Multiplayer
- 12 Performance
- 13 Demo kits
- 14 Troubleshooting and questions
- 15 Lite and Pro

2 Core ideas

IN THIS CHAPTER

- 2.1 Recipes, tracks and steps
- 2.2 Channels, and what happens when effects overlap

2.1 Recipes, tracks and steps

A **recipe** is an asset that defines the feedback for a single moment of play: a hit landing, a heavy landing after a fall, a pickup, a menu button being pressed. A recipe consists of **tracks** arranged on a timeline, in the same manner that a Level Sequence arranges tracks in Sequencer. Each track plays one **step**, which is the effect itself: a camera shake, a screen flash, a sound, a controller vibration or a brief freeze of time. FeelKit Pro has 37 steps and FeelKit Lite has 12 of them; [chapter 16, Step reference](#) describes each step and names its edition.

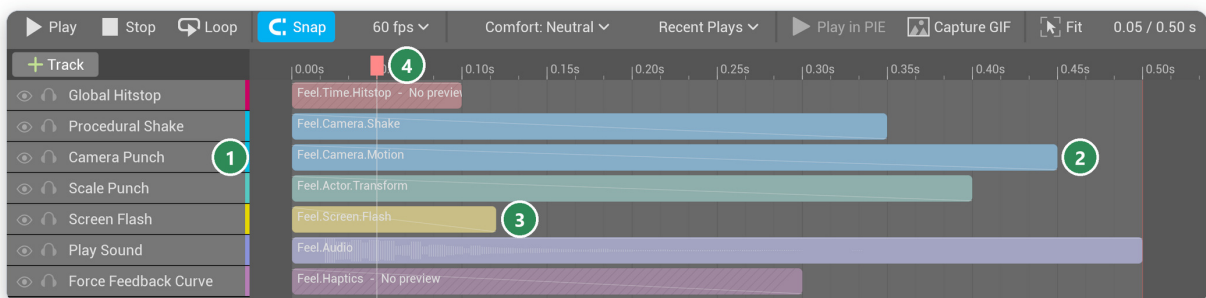


Figure 2.1 The recipe editor on a copy of FR_Impact_HeavyHit. 1 the steps, one per track; 2 a track on the timeline; 3 its channel, by name and color; 4 the playhead

When a recipe plays, each track begins at its **Start Time** and runs its step for its **Duration**, both measured in seconds. A new track lasts 0.5 s. A track with a length of 0 is an instant track: steps that act once, such as Play Sound, Spawn Decal or Blueprint Event, can sit on one. The play ends when its last track ends.

A few settings belong to the recipe as a whole:

- **Cooldown** (default 0 s): how long the same recipe must wait before it can play again on the same target.
- **Max Concurrent** (default 0, which means no limit): how many plays of the recipe can run at once on one target. When the limit is reached, a new play does not start and the running ones continue.
- **Default Intensity** (default 1): a multiplier on every play of the recipe.

The full list of recipe and track fields is in [16.1 Recipe fields](#) and [16.2 Track fields](#).

A step computes its output for a given moment from three inputs only: the time within its track, its intensity at that time, and a seed. It retains no state from earlier frames. This property allows the recipe editor to scrub the timeline in either direction and show exactly the frame the game would show at that moment, and it guarantees that the same seed always produces the same shake. Steps that act once, such as playing a sound or calling a Blueprint event, do so when their track starts, and some act again when it stops.

Recipes are timed in real time, which Unreal refers to as unscaled time. Consequently, a hitstop that slows the world does not slow the recipe that caused it: a shake continues to move during the freeze, and the recipe ends when its timeline specifies. FeelKit's runtime performs work only while a recipe plays or while a change it made still has to be restored; when nothing is playing, it does not tick at all.

2.2 Channels, and what happens when effects overlap

Every track belongs to a **channel**, a gameplay tag under `Feel`. A new track receives its step's default channel, for example `Feel.Camera.Motion` for Camera Punch, and you can change it on the track. The channel colors the track on the timeline, groups the recipe editor's **Intensity** tab, and decides which comfort group scales the track (see [2.7 Comfort on one page](#)).

Channel	Used by	Comfort group
<code>Feel.Camera.Shake</code>	Procedural Shake	Camera Shake
<code>Feel.Camera.Motion</code>	Camera Punch, FOV Kick, Camera Roll, Camera Zoom, Look-at Nudge	Camera Motion
<code>Feel.Time.Hitstop</code>	Global Hitstop, Actor Hitstop	Hitstop and Slow-mo
<code>Feel.Time.SlowMo</code>	Slow-mo Ramp	Hitstop and Slow-mo
<code>Feel.Screen.Flash</code>	Screen Flash	Flashes
<code>Feel.Screen.Distortion</code>	Vignette Pulse, Chromatic Aberration, Post Process Material Pulse	Screen Distortion
<code>Feel.Screen.Color</code>	Desaturate, Color Tint	Screen Distortion

Channel	Used by	Comfort group
Feel.Screen.Fade	Screen Fade	none (Master only)
Feel.Actor.Transform	Scale Punch, Squash and Stretch, Mesh Wobble	none (Master only)
Feel.Actor.Material	Material Parameter Pulse, Hit Flash	none (Master only)
Feel.Actor.Light	Light Flash	Flashes
Feel.Audio	Play Sound	none (Master only)
Feel.Audio.Mix	Sound Class Duck, Pitch Bend, Low-pass Sweep	none (Master only)
Feel.Haptics	Force Feedback Curve, Haptic Pattern	Haptics
Feel.UI	Widget Punch, Widget Shake, Widget Flash, Number Pop	none (Master only)
Feel.Spawn	Spawn Decal, Spawn Particle	none (Master only)
Feel.Meta.Event	Blueprint Event	none (Master only)
Feel.Meta.Recipe	Play Recipe, Random Choice	none (Master only)

The comfort groups in this table are the project defaults. You can change the mapping in the project settings (see [18.5 Comfort](#)).

4 Quick start: your first recipe

IN THIS CHAPTER

- 4.1 Pick a recipe from the library
- 4.2 Preview it without pressing Play
- 4.3 Play it from a Blueprint
- 4.4 Compare with the feel off
- 4.5 Where to go next

This chapter takes one recipe from the FeelKit library into a project, previews it in the recipe editor and plays it on the player's character. It requires no C++ and takes about five minutes. The steps assume that FeelKit is enabled (see 3.2 Installing from Fab and enabling the plugin) and that the project was created from Unreal's Third Person template; any project with a playable character works in the same way.

► **Video: Install FeelKit and play your first recipe (YouTube, about 6 minutes)**

The link is added before release.

4.1 Pick a recipe from the library

FeelKit includes a library of ready-made recipes, grouped by the feeling they produce: Impact, Weight, Power, Speed, Reward, Danger, Dread, Denial and Interface. Library recipes are read-only, so that an update of FeelKit can replace them without affecting a project; a project always works with its own copy. This chapter uses `FR_Impact_HeavyHit`, a heavy blow that combines a short freeze, a camera shake, a camera punch, a scale punch, a flash, a sound and a controller rumble.

1. In the Content Browser, open **Settings** and turn on **Show Plugin Content**. FeelKit's content then appears under **Plugins > FeelKit Content**.
2. Open **FeelKit Content > Library > Impact** and select `FR_Impact_HeavyHit`.

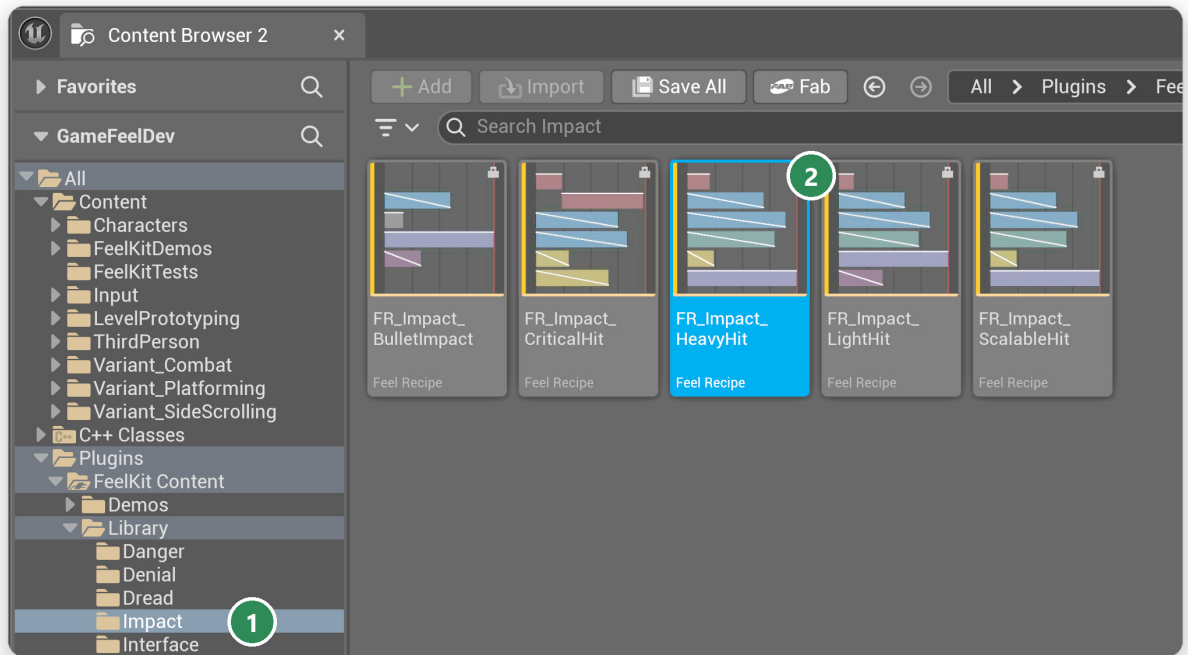


Figure 4.1 The Impact folder of the FeelKit library in the Content Browser. 1 the folder; 2 FR_Impact_HeavyHit

3. Double-click the recipe. The recipe editor opens it with a banner across the timeline: "Library recipe (read-only). Copy it to your project to edit."
4. Click **Copy to Project**. In the save dialog, choose a folder of the project, keep the suggested name **HeavyHit** and click **Save**. The copy opens in the recipe editor and can be changed freely; its **Based On** field records the library recipe it came from.

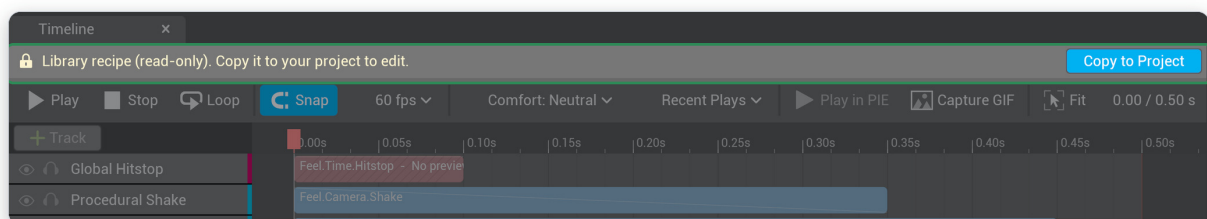


Figure 4.2 A library recipe opens read-only. Copy to Project creates an editable copy in the project

Note In FeelKit Pro, **Tools > FeelKit Recipe Browser** presents the same library with filters by feeling, genre and channel, and plays each recipe when the pointer rests on it. Chapter 9, The recipe library and browser describes it.

4.2 Preview it without pressing Play

The recipe editor plays a recipe on its own preview camera and mesh, using the same code as the game (see 2.8 [Why the editor preview matches the game](#)). Most of a recipe can therefore be judged without starting the game.

1. In the **Timeline** toolbar, click **Play**, or press Space while the timeline has focus. The preview shows the camera shake, the camera punch, the scale punch and the flash, and the sound plays.
2. Click **Loop** to repeat the recipe while observing it.
3. Drag the red playhead along the ruler to step through the recipe. Because steps retain no state between frames, scrubbing backwards shows exactly the same frames as playing forwards.

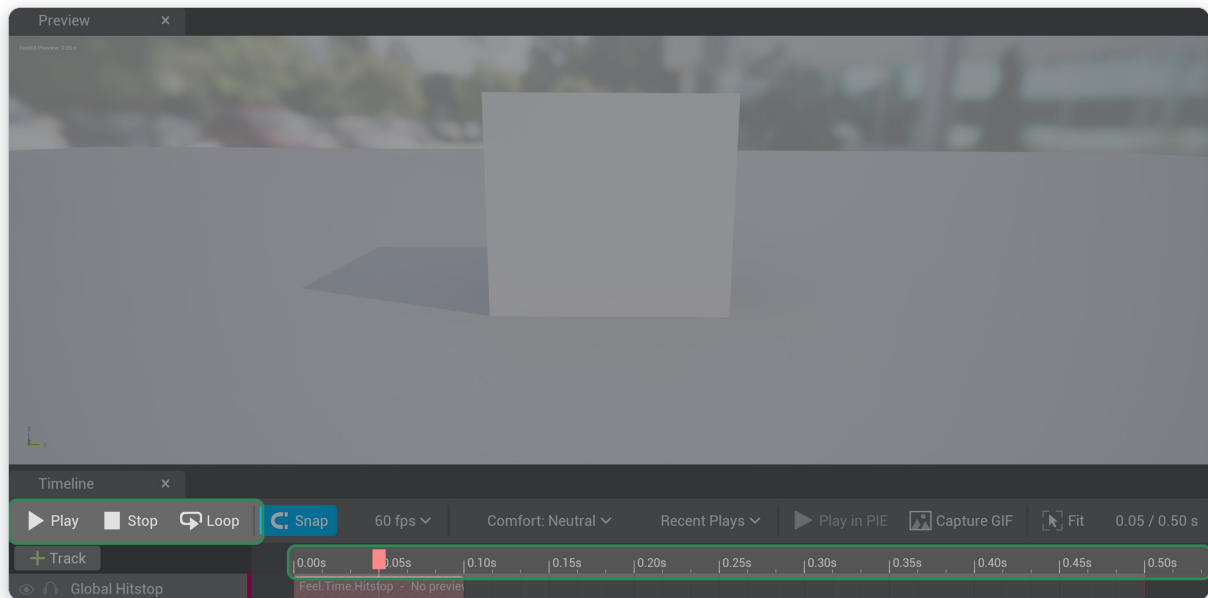


Figure 4.3 The recipe editor's preview above the Timeline toolbar. Play, Stop and Loop on the left; the ruler for scrubbing on the right

Two tracks of `HeavyHit` carry the label **No preview**: Global Hitstop, which slows the game's time, and Force Feedback Curve, which drives the controller. Both act only in a running game, which the next section starts.

4.3 Play it from a Blueprint

A recipe plays when the game calls the **Play Feel** node. The following steps play `HeavyHit` on the player's character whenever the 1 key is pressed.

1. Open the character Blueprint, for example `BP_ThirdPersonCharacter`, and its **Event Graph**.
2. Right-click an empty area of the graph, type **1** and choose **Input > Keyboard Events > 1**.
3. Drag from the **Pressed** pin, type **Play Feel** and choose **Play Feel** from the **Feel** category.
4. On the Play Feel node, set **Recipe** to `HeavyHit`.
5. Drag from the **Target** pin and choose **Make Feel Target from Actor**. Right-click the graph, choose **Get a reference to self**, and connect it to the **Actor** pin.
6. Compile and save the Blueprint.

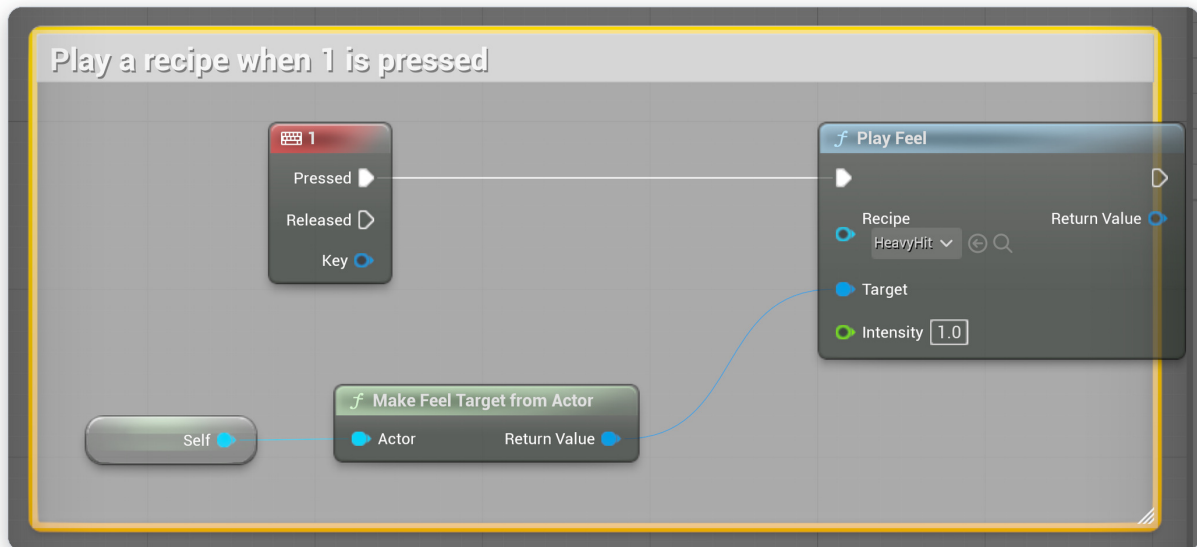


Figure 4.4 The Event Graph after step 5: the 1 key plays HeavyHit, with the character itself as the target

Start Play In Editor and press 1. The recipe plays on the character: the camera kicks and shakes, the screen flashes, the character's mesh punches in scale, time freezes briefly and a connected controller rumbles.

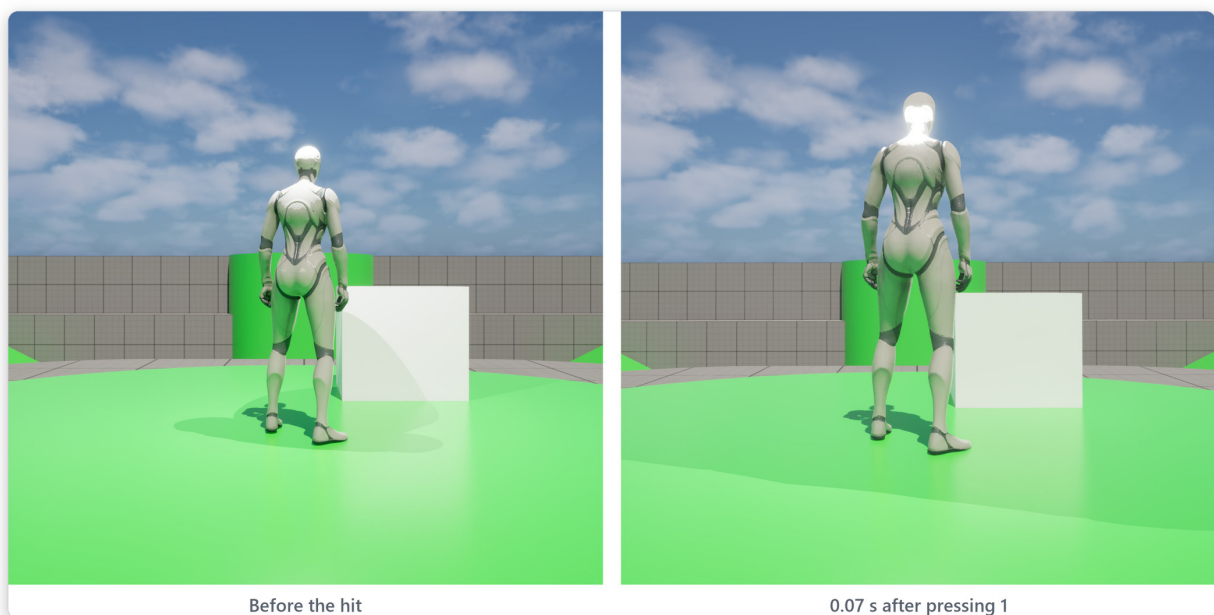


Figure 4.5 The same view before the hit and 0.07 s after pressing 1, when the camera punch and the scale punch are at their strongest. Motion blur was turned off for this picture

The target decides where each effect is applied (see [2.4 Targets: what a recipe plays on](#)). Camera, screen and controller effects reach the player who owns the target, here the player controlling the character. Actor effects apply to the character; because a character's root is its collision capsule, FeelKit moves and scales the visible mesh instead, so collision and movement are never affected.

A keyboard event in the character Blueprint is a quick way to try a recipe. In a finished game, recipes are played from the game's own events, such as a hit being registered or a landing; Chapter 6, Playing recipes in your game describes the options, including triggers that need no Blueprint at all.

4.4 Compare with the feel off

The **Feel Switch** turns FeelKit off and on while the game runs, so that the difference can be seen and felt directly. It is useful for tuning, for playtests and for showing a game to others.

1. Open **Window > Place Actors**, search for **Feel Switch** and drag it into the level.
2. Start Play In Editor. A start card explains the switch, and a **FEEL: ON** badge appears in the top right corner, with the key for the comfort settings below it.



Figure 4.6 The Feel Switch at the start of play: the start card and the FEEL: ON badge

3. Press 1 to play the recipe, then press Tab (View / Share on a controller). The badge changes to **FEEL: OFF**, and pressing 1 now shows the game without FeelKit. Press Tab again to turn FeelKit back on.



Figure 4.7 The badge while FeelKit is turned off

Turning the feel off stops only FeelKit: effects that the game itself plays are unaffected. Every play session starts with FeelKit on, and the choice is not saved. Pressing O (Menu / Options on a controller) opens the comfort settings menu, which Chapter 8, Comfort and accessibility describes. The switch keys, the start card and the controls list are set in the Feel Switch's details.

4.5 Where to go next

- Chapter 5, The recipe editor explains every part of the recipe editor, including how to build a recipe from an empty timeline.
- Chapter 6, Playing recipes in your game shows how to play recipes from gameplay events, animations and triggers.
- Chapter 7, Parameters, context and variation covers parameters, context and variation, which let one recipe serve light and heavy hits alike.
- Chapter 8, Comfort and accessibility covers the comfort settings and the comfort menu for players.
- [Chapter 16, Step reference](#) lists every step with its settings.

PART TWO

Reference

- 16 Step reference
- 17 Blueprint nodes and components
- 18 Project settings and editor preferences
- 19 Console commands
- 20 Writing your own steps in Blueprint and C++
- 21 Recipe JSON files and editor scripting
- 22 Credits and licences
- 23 Version history

16 Step reference

16.5 Camera steps

Camera steps offset the view of the target's local player, through a camera modifier that FeelKit adds to each local player's camera manager. Tracks of one play add up; separate plays are combined by the **Camera Arbitration** setting (see [18.2 Camera](#)).

16.5.1 Procedural Shake

Moves and turns the camera with noise or a sine wave computed from the track time and the seed, so the same seed gives the same shake every time and the preview shows exact frames when scrubbing. **Perlin** gives smooth random motion on each axis. **Sine** gives a regular oscillation with a random phase per axis. **Directional** moves back and forth along **Direction** only, with the rotation swinging in step. The intensity curve sets the strength over time; the default curve gives a shake that dies away over the track.

Edition	Lite and Pro
Class	UFeelStep_ProceduralShake, module FeelCore.
Default channel	Feel.Camera.Shake.
Comfort group	Camera Shake.
Editor preview	yes.
Track length	needs a length.
New tracks start with	an intensity curve that fades from 1 to 0.

Property	Default	Description
Mode	Perlin	Shape of the shake.
Frequency	12 Hz	Oscillations per second. At least 0.01.
Location Amplitude	(0, 3, 3) cm	Maximum location offset per camera axis (X forward, Y right, Z up). Shown only when Mode is not Directional .

Property	Default	Description
Direction	(0, 0, 1)	Camera-space direction of a directional shake. Shown only when Mode is Directional .
Directional Amplitude	8 cm	Maximum offset along Direction. Shown only when Mode is Directional .
Rotation Amplitude	Pitch 1, Yaw 1, Roll 0.5 (degrees)	Maximum rotation offset in degrees per axis.
Field Of View Amplitude	0°	Maximum field of view change.

16.5.2 Camera Punch

Pushes and turns the camera to a peak and back along the motion shape; with the default Spring it overshoots and settles like a physical hit. **Location Punch** and **Rotation Punch** are the offsets at the peak. **Direction Source** can point the punch along the play's Direction, or away from or toward the play's Location, as seen from the camera when the play started. The strength stays that of the step settings, and the step settings are used when the play passed no direction or location. **Direction Jitter** turns the direction by a random angle for each play, which stays fixed for the whole play.

Edition	Lite and Pro
Class	UFeelStep_CameraPunch, module FeelCore.
Default channel	Feel.Camera.Motion.
Comfort group	Camera Motion.
Editor preview	yes.
Track length	needs a length.
New tracks start with	a flat intensity curve at 1.
Motion shape	Spring by default; the shape settings are described in 16.4 Shared motion settings .

Property	Default	Description
Location Punch	(-10, 0, -4) cm	Camera-space location offset at the peak (X forward, Y right, Z up).
Rotation Punch	Pitch -2.5, Yaw 0, Roll 0 (degrees)	Rotation offset at the peak, in degrees.
Direction Source	Step Settings	Where the punch direction comes from. With a play direction or location, the punch keeps the strength of Location Punch and Rotation Punch but points along that direction, as seen from the camera when the play started.
Direction Jitter	0°	Varies the punch direction per play by up to this angle, so repeated plays never look identical. Range 0 to 90.

16.5.3 FOV Kick

Changes the field of view by **Field Of View Kick** at the peak and back. Positive values widen the view, which reads as speed; negative values narrow it, which reads as an impact zoom. The default Kick shape rises fast and eases back.

Edition	Lite and Pro
Class	UFeelStep_FOVKick, module FeelCore.
Default channel	Feel.Camera.Motion.
Comfort group	Camera Motion.
Editor preview	yes.
Track length	needs a length.
New tracks start with	a flat intensity curve at 1.
Motion shape	Kick by default; the shape settings are described in 16.4 Shared motion settings .

Property	Default	Description
Field Of View Kick	8°	Field of view change at the peak. Positive widens (speed), negative narrows (impact zoom). Range -60 to 60.

16.5.4 Camera Roll PRO

Rolls the camera around its view axis to **Roll Degrees** at the peak and back. With **Random Direction** on, each play picks clockwise or counterclockwise from its seed. Players who turn **Allow Camera Roll** off in their comfort settings see no camera roll from any recipe.

Edition	Pro only
Class	UFeelStep_CameraRoll, module FeelCore.
Default channel	Feel.Camera.Motion.
Comfort group	Camera Motion; the roll is also removed completely when a player turns Allow Camera Roll off.
Editor preview	yes.
Track length	needs a length.
New tracks start with	a flat intensity curve at 1.
Motion shape	Spring by default; the shape settings are described in 16.4 Shared motion settings .

Property	Default	Description
Roll Degrees	4°	Roll angle at the peak. Positive rolls clockwise. Range -45 to 45.
Random Direction	On	Pick clockwise or counterclockwise at random for each play.